Assignment 2 - nanoGPT Data Parallelism on Shakespeare

Daize Dong . Generated 2025-10-06 14:22

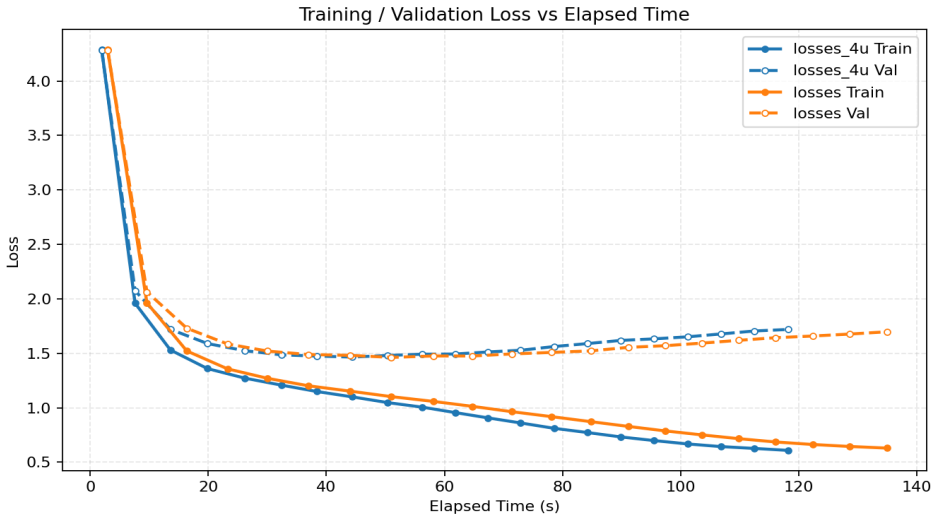
Global Settings

- Dataset: Shakespeare (vocabulary size 65)
- Model: nanoGPT (~10.65M parameters)
- Optimizer: AdamW
- Global batch size: 64 for Experiments A, B, and D; 64/128/256/512/1024 for Experiment C
- Sequence length: 256
- Gradient accumulation: 1
- Fixed random seeds for reproducibility
- Cluster (DeltaAI): 4 GH200 GPUs on each node

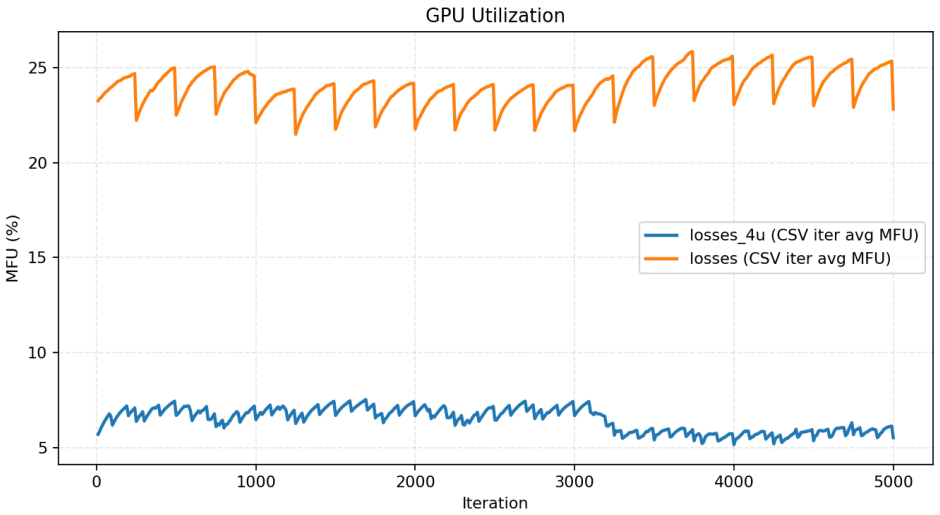
Experiment A - Fixed Global Batch: 1 GPU vs 4 GPUs

Experiment-specific settings

- Devices: 1 GPU vs 4 GPUs on a single node, data parallel
- Per-GPU microbatch on GPUs: 16



Loss curve by time (s)



GPU utilization (%) by steps

Findings

- The 4-GPU run reaches the same loss in fewer seconds (left-shifted time-loss), but the speedup is less than 4x.
- The validation loss matches the 1-GPU run when compared at the best checkpoint.

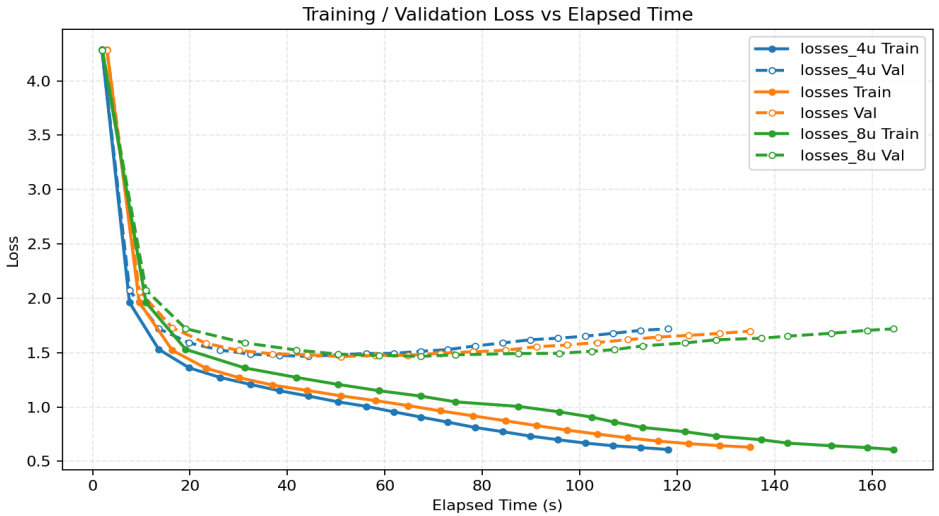
Analysis

- Small per-GPU microbatches increase the fraction of communication overheads, resulting in sublinear scaling. 4 GPUs achieve only ~1.2x speedup over 1 GPU in wall-clock time.
- The 4-GPU run shows lower GPU utilization, indicating that GPUs are not compute-bound and all-reduce and launch latencies dominate a larger share of step time.

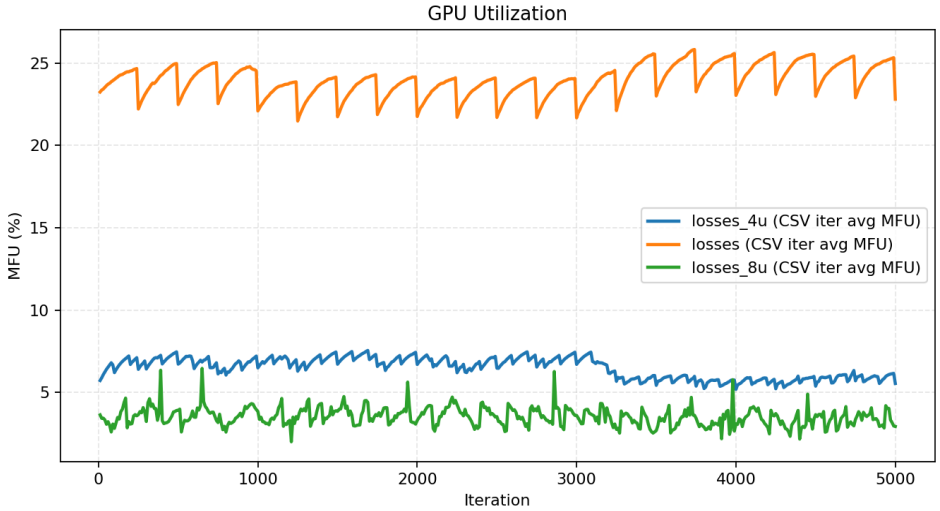
Experiment B - Fixed Global Batch: 1 GPU vs 4 GPUs vs 8 GPUs

Experiment-specific settings

- Devices: 1 GPU vs 4 GPUs (1 node) vs 8 GPUs (2 nodes x 4 GPUs)
- Per-GPU microbatch on 8 GPUs: 8



Loss curve by time (s)



GPU utilization (%) by steps

Findings

- The 8-GPU (multi-node) run fails to beat 4 GPUs and can even trail the 1-GPU baseline in wall-clock time.
- GPU utilization on 8 GPUs is lower and more erratic.

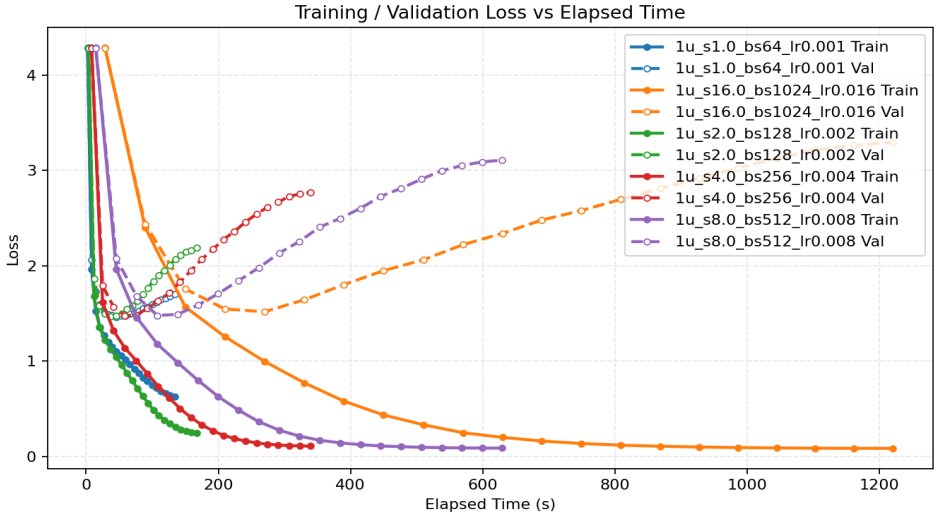
Analysis

- Inter-node bandwidth and latency on DeltaAI are inferior to intra-node NVLink, making multi-node scaling inefficient.
- The 8-GPU run shows lower GPU utilization, indicating that GPUs are not compute-bound.

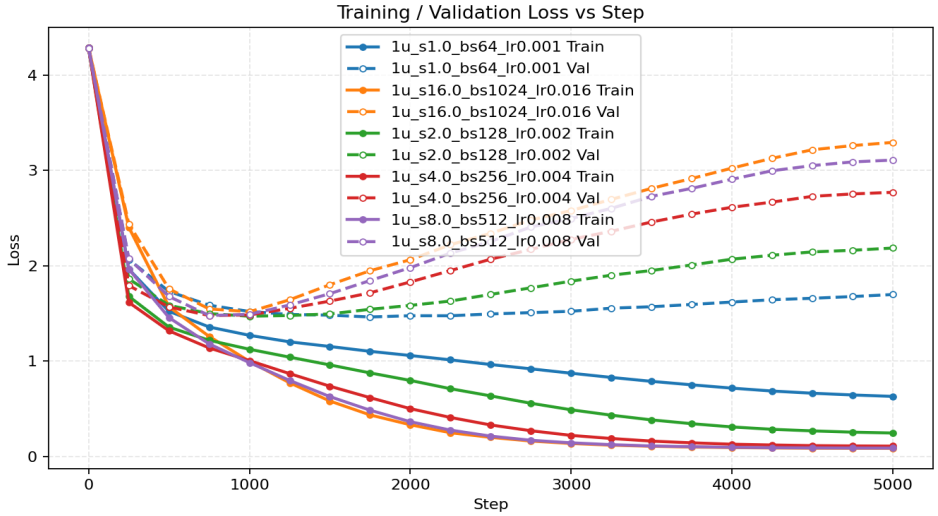
Experiment C - 1 GPU Batch Scaling

Experiment-specific settings

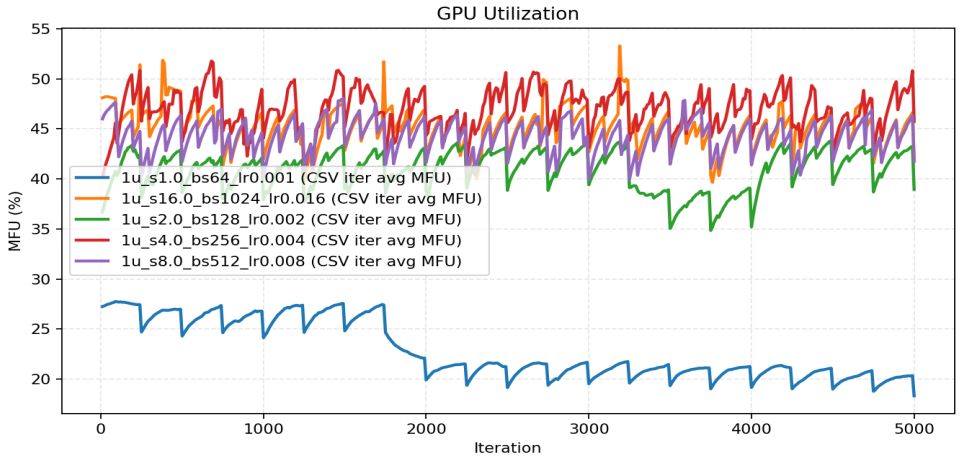
- Devices: 1 GPU
- Per-GPU microbatch: 64
- Increase per-step batch (64 -> 128 -> 256 -> 512 -> 1024)
- I went beyond from the requested 4x to 16x!!!



Loss curve by time (s)



Loss curve by steps



GPU utilization (%) by steps

Findings

- Increasing batch size lifts utilization and shifts the time-loss curve left-optimization advances faster per minute.
- The best validation loss remains essentially unchanged across batch sizes.

Analysis

- Larger batches improve arithmetic intensity and kernel efficiency, reducing the pipeline overheads.
- Since data and model are held constant, the attainable optimum (validation) is similar.

Experiment D - Four GPUs with Learning-Rate Warmup

Experiment-specific settings

- Devices: 4 GPUs
- Per-GPU microbatch: 32
- Linear warmup for the first 100 steps vs no warmup (baseline)

Loss curve by steps

Findings

- Learning-rate warmup smooths the first few hundred steps and lead s to a slightly better final loss.

Analysis

- Early oversized updates can overshoot before AdamW's moments settle down, and warmup mitigates this risk.